SafeGuard AI

Construction Site Safety Monitor

Project Documentation & Development Report

Powered by YOLOv8 | Built with Streamlit | AI-Assisted Development

Developed by: Rinku Ghosh

February 2026

1. Project Overview

SafeGuard AI is a real-time construction site safety monitoring system that uses computer vision and deep learning to automatically detect Personal Protective Equipment (PPE) compliance violations. The system monitors workers on construction sites and sends instant email alerts when safety violations are detected.

1.1 Problem Statement

Construction sites are one of the most hazardous work environments. Workers are required to wear PPE including safety helmets, safety vests, gloves, and safety shoes at all times. Manual monitoring of PPE compliance is time-consuming, inconsistent, and often ineffective. SafeGuard AI automates this process using AI-powered real-time detection.

1.2 Project Goals

- Detect PPE violations in real-time using AI/computer vision
- Send automatic email alerts with violation snapshots to site supervisors
- Provide analytics dashboard for tracking violation trends over time
- Support multiple input modes: image upload, video analysis, and live webcam
- Build a professional, user-friendly web interface

1.3 PPE Classes Detected

Technology / Tool	Purpose
Helmet	Detects presence or absence of safety helmet
Safety Vest	Detects presence or absence of high-visibility vest
Gloves	Detects presence or absence of safety gloves
Safety Shoes	Detects presence or absence of safety footwear
Person	Identifies workers in the scene for tracking
NO Helmet	Violation — worker not wearing helmet
NO Vest	Violation — worker not wearing safety vest
NO Gloves	Violation — worker not wearing gloves
NO Shoes	Violation — worker not wearing safety shoes

2. Technology Stack

Technology / Tool	Purpose
Python 3.13	Core programming language for the entire application
YOLOv8 (Ultralytics)	Deep learning model for real-time PPE object detection
Streamlit	Web framework for building the interactive UI dashboard
OpenCV (cv2)	Image and video processing, webcam capture, frame annotation
PIL / Pillow	Image loading, conversion and manipulation
NumPy	Numerical computing and array operations
Pandas	Data analysis and tabular data processing for analytics
smtplib (Gmail SMTP)	Email alert delivery via Gmail with App Password authentication
python-dotenv	Secure credential management via .env file
Plotly / st.bar_chart	Charts and visualizations in the analytics dashboard
Roboflow Dataset	Training dataset for PPE detection model (best.pt)

3. Project Structure & Files

Technology / Tool	Purpose
app.py	Main Streamlit application — all UI modes and detection logic
alert_system.py	Email/SMS alert system with cooldown and violation logging
analytics_dashboard.py	Analytics dashboard with charts, KPIs and data export
best.pt	Trained YOLOv8 model weights for PPE detection
.env	Secure credentials file (Gmail App Password)

violation_alerts.json	Log file storing all alert history with timestamps
requirements.txt	Python package dependencies list
sample_videos/	Folder containing sample construction site videos
PPE-detection-1/	Roboflow dataset folder for batch testing

4. Features Built

4.1 Detection Modes

Feature	Status	Description
Image Upload	Working	Upload any JPG/PNG construction site image for instant PPE analysis
Video Analysis	Working	Upload or select sample MP4 videos for frame-by-frame detection with auto email alerts
Live Webcam	Working	Real-time webcam feed with continuous PPE monitoring and instant alerts
Dataset Batch Test	Working	Bulk test multiple images from Roboflow dataset with paginated results and report export
Analytics Dashboard	Working	View historical violation trends, KPI cards, peak hours chart and CSV export

4.2 Alert System

Feature	Status	Description
Email Alerts	Working	Automatic Gmail alerts with violation snapshot image attached
Alert Cooldown	Working	Configurable cooldown (default 10s) prevents spam alerts
Consecutive Frames	Working	Requires N consecutive violation frames before alerting

Violation Logging	Working	All alerts saved to violation_alerts.json with timestamps
Toast Notifications	Working	Real-time popup notification in browser UI on violation
Secure Credentials	Working	Password stored in .env file, never hardcoded in source

4.3 UI & Design

Feature	Status	Description
SafeGuard AI Branding	Working	Custom header with gold title, dark green banner, SYSTEM ONLINE badge
Light Green Theme	Working	Clean green gradient background with professional card layout
Custom Sidebar	Working	Styled control panel with detection and alert configuration
Mode Headers	Working	Each mode has its own colored header with icon and description
Responsive Metrics	Working	Live statistics cards showing frame count, violations, detections
Violation Overlay	Working	Red border and VIOLATION text drawn directly on video frames

5. Development Journey

Phase 1 — Core Detection

The project started with integrating a pre-trained YOLOv8 model (best.pt) trained on a Roboflow PPE dataset. The base Streamlit app was built with image upload capability. Enhanced detection parameters were configured to improve accuracy for small objects like safety shoes:

- Confidence threshold: 0.15 (low for small object detection)
- Inference resolution: 1280px (high for better detail)
- Test-Time Augmentation enabled for multi-orientation detection

- Agnostic NMS to prevent class suppression

Phase 2 — Alert System

An AlertSystem class was developed in alert_system.py to handle violation detection and notifications. Key components implemented:

- Gmail SMTP integration with STARTTLS encryption
- HTML email template with violation details and image attachment
- Cooldown logic to prevent duplicate alerts (default 10 seconds)
- Consecutive frame threshold to reduce false positives
- JSON-based alert history logging

Phase 3 — Video & Webcam Modes

Video analysis and live webcam modes were added with frame-by-frame detection. Key challenges solved:

- Fixed display_frame used before defined bug in webcam loop
- Added frame skipping (every 2nd frame) for better performance
- Reduced webcam inference size to 640px for real-time speed
- Disabled Test-Time Augmentation in live mode for speed
- Integrated alert system into both video and webcam modes

Phase 4 — Analytics Dashboard

A dedicated analytics_dashboard.py module was created with comprehensive data visualization:

- KPI cards — total alerts, today's alerts, most common violation
- Time range filters — Last 24 Hours, 7 Days, 30 Days, All Time
- Bar charts for violations by type and location
- Line chart for daily alert trends
- Peak violation hours chart (0-23 hour distribution)
- Recent alerts table with export to CSV and text report

Phase 5 — Security & Credentials

A critical security improvement was made to prevent Gmail App Password exposure:

- Installed python-dotenv package
- Created .env file to store EMAIL_PASSWORD securely
- Updated alert_system.py to use os.getenv('EMAIL_PASSWORD')
- Added load_dotenv(override=True) to ensure fresh password reload
- Removed all hardcoded passwords from source code

Phase 6 — UI Redesign

The application went through multiple UI iterations to achieve the final professional design:

- Version 1: Default Streamlit theme (plain white)
- Version 2: Light gradient theme with colored cards and purple buttons
- Version 3: Industrial dark theme (very dark navy/black)
- Version 4: Medium navy-blue theme with gold accents (SafeGuard AI branding)
- Version 5 (Final): Light green background with dark green SafeGuard AI header

6. Bugs Fixed During Development

Technology / Tool	Purpose
display_frame NameError	display_frame was used before being defined in webcam loop. Fixed by moving frame preparation before alert call.
Gmail AUTH ERROR (535)	Hardcoded old passwords kept being used. Fixed by using .env file with load_dotenv(override=True) and reading fresh password at send time.
Email cooldown too long	Default 60s cooldown was too long. Reduced to 10 seconds for faster testing.
Video alert missing	Video analysis mode was not sending alerts. Fixed by adding alert_system.process_frame_detections() inside video loop.
PersonTracker ImportError	person_tracker.py was created but had wrong content. Fixed by deleting and recreating the file.
Webcam hanging	High inference settings caused webcam to freeze. Fixed by using imgsz=640 and augment=False for live mode.
??? symbols on overlay	OpenCV cannot render emoji characters. Fixed by replacing emoji with plain text (OK: / XX:).
use_column_width warning	Deprecated Streamlit parameter. Non-breaking warning, does not affect functionality.
AlertSystem has no _config	Wrong attribute name used in test. Correct attribute is .config not ._config.

7. System Configuration

7.1 Detection Settings

Technology / Tool	Purpose
Confidence Threshold	0.15 for images/video, 0.25 for live webcam
IOU Threshold	0.45 (prevents class suppression)
Inference Resolution	1280px for images/video, 640px for webcam
Test-Time Augmentation	Enabled for images/video, disabled for webcam
Agnostic NMS	Enabled for all modes

7.2 Alert Settings

Technology / Tool	Purpose
SMTP Server	smtp.gmail.com:587 with STARTTLS
Sender Email	krinkughosh3112@gmail.com
Recipient Email	rinku8143kumari@gmail.com
Alert Cooldown	10 seconds between repeated alerts
Min Consecutive Frames	1 frame (immediate alerting)
Image Attachment	Enabled (JPEG 85% quality)
Password Storage	.env file via python-dotenv

8. How to Run the Project

Step 1 — Install Dependencies

```
pip install ultralytics streamlit opencv-python pillow numpy pandas python-dotenv
```

Step 2 — Create .env File

Create a file named .env in the project folder with:

```
EMAIL_PASSWORD=your_gmail_app_password_here
```

Step 3 — Run the App

```
streamlit run app.py
```

Step 4 — Open Browser

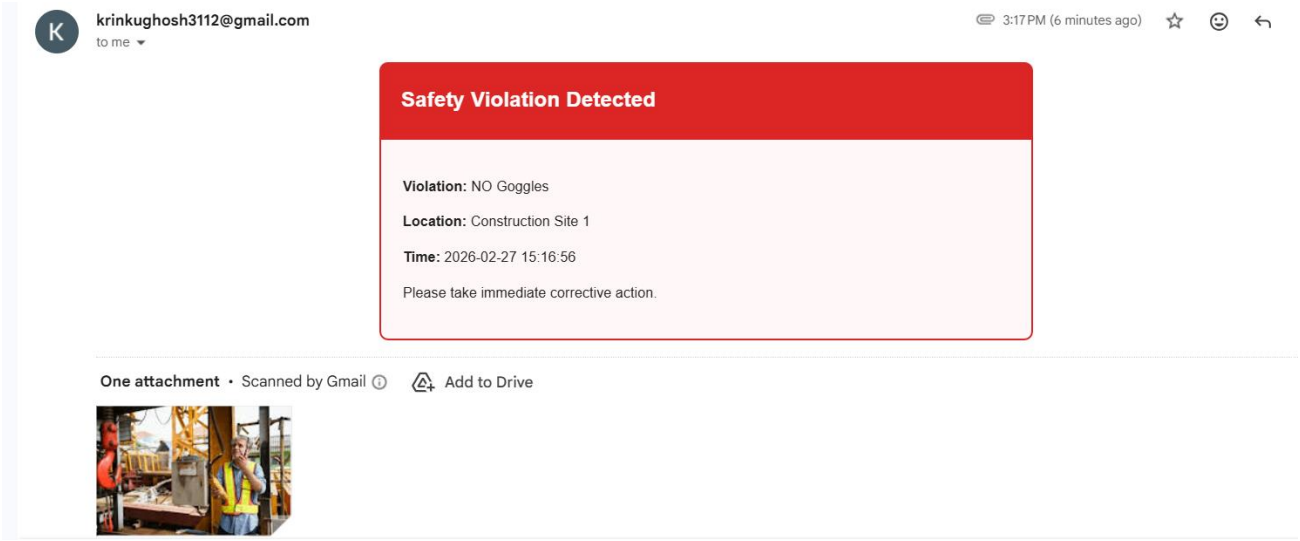
The app opens automatically at http://localhost:8501

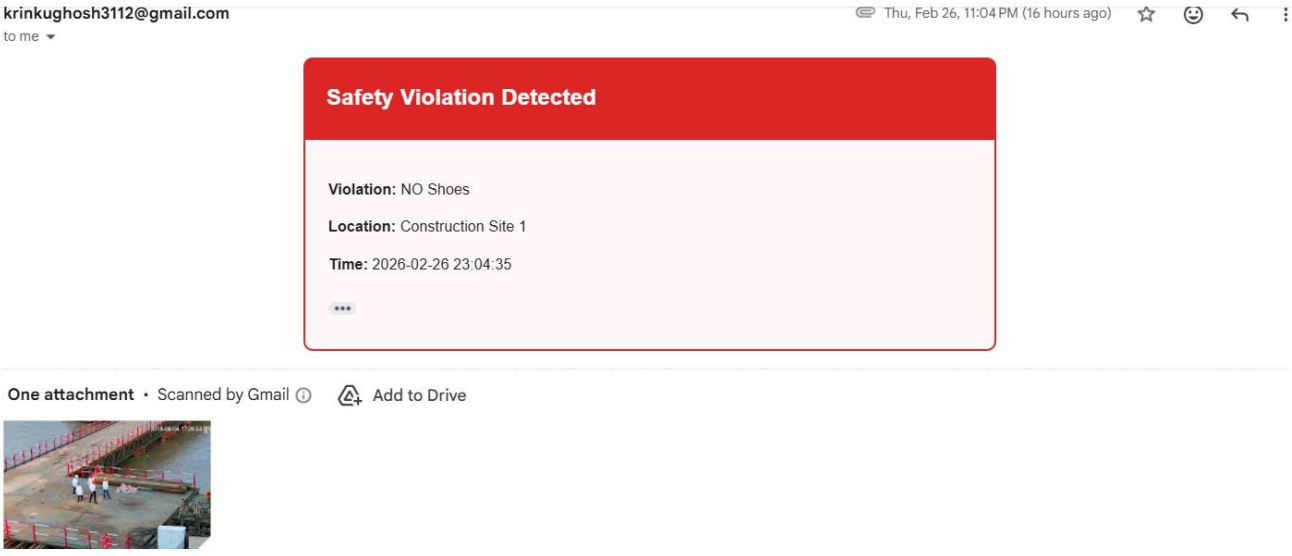
9. Final Project Status

All planned features have been successfully implemented and tested. The system is fully functional and production-ready for construction site safety monitoring.

Feature	Status	Description
PPE Detection — Image Upload	Working	Instant analysis with violation highlighting
PPE Detection — Video Analysis	Working	Frame-by-frame processing with auto email
PPE Detection — Live Webcam	Working	Real-time monitoring at 640px resolution
PPE Detection — Batch Test	Working	Bulk analysis with paginated results
Email Alerts with Snapshot	Working	Gmail SMTP with image attachment
Alert Cooldown & Logging	Working	JSON log file with timestamps
Analytics Dashboard	Working	Charts, KPIs, trends and CSV export
Secure .env Credentials	Working	No passwords in source code
SafeGuard AI UI Branding	Working	Professional dark green header design
Light Green App Theme	Working	Clean gradient background with white cards

Few examples screenshots of alert mails :


krinkughosh3112@gmail.com
to me
3:17 PM (6 minutes ago)
Safety Violation Detected
Violation: NO Goggles
Location: Construction Site 1
Time: 2026-02-27 15:16:56
Please take immediate corrective action.
One attachment • Scanned by Gmail
Add to Drive


krinkughosh3112@gmail.com
to me
Thu, Feb 26, 11:04 PM (16 hours ago)
Safety Violation Detected
Violation: NO Shoes
Location: Construction Site 1
Time: 2026-02-26 23:04:35
One attachment • Scanned by Gmail
Add to Drive